
Course	: Diploma in Electronic Systems (EGDF18) : Diploma in Electronic & Computer Engineering (EGDF20)
Module	: Embedded System Design & Technology (EGE320)
Laboratory No.	: Lab 6
Laboratory Title	: Digital Output
Objective	: To illustrate the concept of digital output. : To demonstrate the use of GPIO peripherals. : To develop application in C using both GPIO peripherals.

Contents

1. RGB LED Interface Circuit
 - 1.1. Multicolor LED Display
 - 1.2. Torch Light App
 - 1.3. Integrating Torch Light App

1 RGB LED Interface Circuit

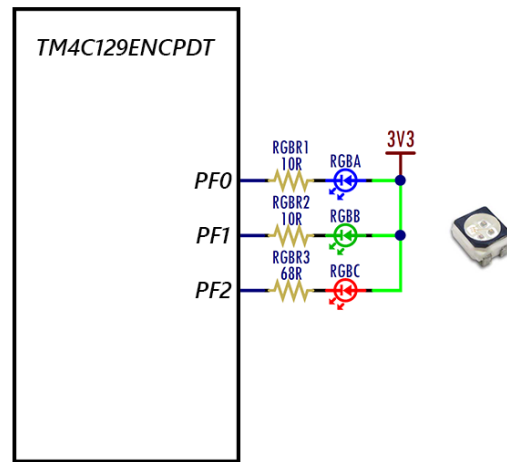


Figure 1a: RGB LED Interface Circuit

1.1 Multicolor LED Display

1. Setup the CCS IDE development environment learned in Lab1.
2. To control the digital output connected with the RGB LED, the following function is needed.

```
void RGBLEDOnOff(bool bRed, bool bGreen, bool bBlue)
```

1 or 0 to ON or OFF the RED LED

1 or 0 to ON or OFF the GREEN LED

1 or 0 to ON or OFF the BLUE LED

3. Complete the code below while entering it into CCS IDE. Build to ensure no errors in the code.

```
#include <stdbool.h>
#include <stdint.h>
#include "clock.h"
#include "rgbled.h"                                //Include the rgbled driver file

uint32_t ui32Count = 0;

void MultiColorLEDDisplay(void)
{
    //Turn ON Red LED
    RGBLEDOnOff(1, 0, 0);

    //LED ON for ~0.5s
    for(ui32Count=0; ui32Count<3000000; ui32Count++);

    //Turn OFF Red LED
    RGBLEDOnOff(0, 0, 0);

    //LED OFF for ~0.5s
    for(ui32Count=0; ui32Count<3000000; ui32Count++);

    //Turn ON and OFF Green LED
    RGBLEDOnOff(0, 1, 0);
    for(ui32Count=0; ui32Count<3000000; ui32Count++);
    RGBLEDOnOff(0, 0, 0);
    for(ui32Count=0; ui32Count<3000000; ui32Count++);

    //Turn ON and OFF Blue LED
    RGBLEDOnOff(0, 0, 1);
    for(ui32Count=0; ui32Count<3000000; ui32Count++);
    RGBLEDOnOff(0, 0, 0);
    for(ui32Count=0; ui32Count<3000000; ui32Count++);
}

int main(void)
{
    ClockFrequencySet(120000000);                    //Set processor clock to 120MHz
    RGBLEDPinInit();                                //Initialize RGB LED pin

    while(1)
    {
        //MultiColorLEDDisplay function call
        MultiColorLEDDisplay();
    }
}
```

4. Debug and run the code using F8 key.

5. Observed that the RGB LED is lighting up with Red, green and blue color showing one at a time.

1.2 Torch Light App

1. The final user interface design and touch action to be performed is shown in Figure below.

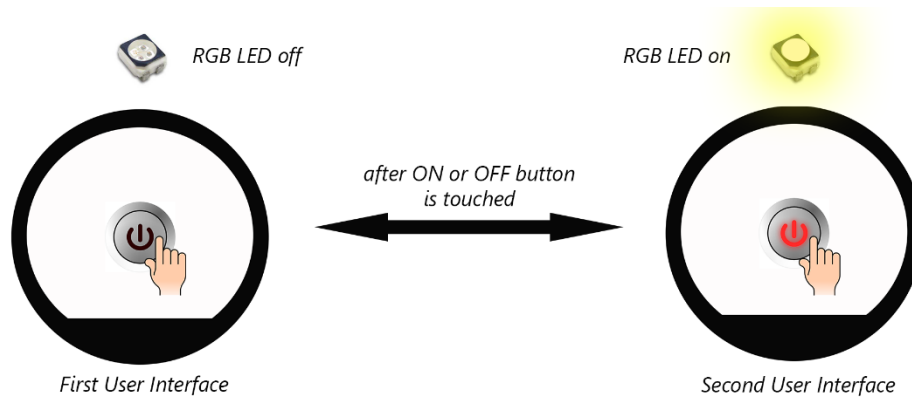


Figure 1.2a: Torch Light App

2. Setup the CCS IDE development environment learned in Lab1.
3. Download the On.c and Off.c from blackboard. Copy and paste the file in the project as shown in Figure below.

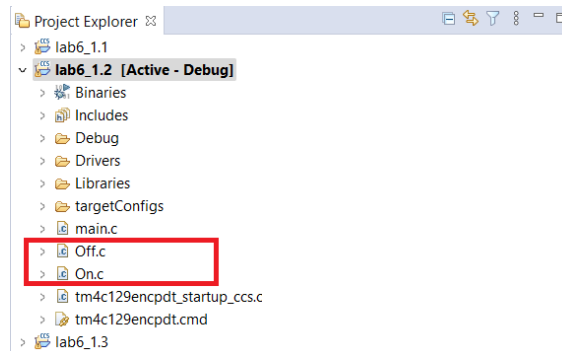


Figure 1.2b: Location of files

4. Complete the code below while entering it into CCS IDE. Build to ensure no errors in the code.

```
#include <stdbool.h>
#include <stdint.h>
#include "clock.h"
#include "glib.h"
#include "st7789_spi.h"           //Include the display driver file
#include "it7258_i2c.h"          //Include the touch driver file
#include "rgbled.h"              //Include the rgbled driver file

extern const uint8_t g_pui8On[];
extern const uint8_t g_pui8Off[];

tContext sContext = {};
uint16_t ui16XPosition[3] = {};
uint16_t ui16YPosition[3] = {};
uint8_t ui8Pressure[3] = {};
bool bTouchStatus = 0;

void TorchLightApp(void)
{
    bool bLEDStatus = 0;

    //Print Off image
    GrImageDraw(&sContext, g_pui8Off, 80, 80);

    while(1)
    {
        if(bTouchStatus == 1)
        {
            //Read Touch Point
            IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);

            if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
            {
                if(((100 < ui16XPosition[0]) && (ui16XPosition[0] < 140)) &&
                    ((100 < ui16YPosition[0]) && (ui16YPosition[0] < 140)))
                {
                    //Check if LED is On or Off
                    if (bLEDStatus == 0)
                    {
                        //Turn ON all LEDs
                        _____;

                        //Print On image
                        GrImageDraw(&sContext, g_pui8On, 80, 80);

                        //Set bLEDStatus to 1
                        _____;
                    }
                    else
                    {
                        //Turn OFF all LEDs
                        RGBLEDOnOff(0, 0, 0);

                        //Print Off image
                        _____;

                        //Clear bLEDStatus to 0
                        _____;
                    }
                }
            }
        }
    }
}
```

```

        //Wait for touch to be released
        while((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
        {
            IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
        }

        //Reset bTouchStatus
        bTouchStatus = 0;
    }
}

void TouchPanelIntHandler(void)           //Touch Panel ISR
{
    IT7258IntClear();                     //Clear Interrupt
    bTouchStatus = 1;                     //Set bTouchStatus
}

int main(void)
{
    ClockFrequencySet(120000000);          //Set processor clock to 120MHz
    ST7789PinInit(120000000);              //Initialize LCD Controller pin
    IT7258PinInit(120000000);              //Initialize Touch Controller pin
    RGBLEDPinInit();                       //Initialize RGBLED pin

    GrContextInit(&sContext, &g_sST7789);  //Initialize glib Context
    IT7258IntRegister(TouchPanelIntHandler); //Register Touch Panel Interrupt

    TorchLightApp();                       //TorchLightApp Function Call
}

```

5. Debug and run the code using F8 key.
6. Touch on the On and Off button at the center of the LCD screen and observed the changes on the RGB LED as shown in Figure 1.2a.

1.3 Integrating Torch Light App

1. The flow of the user interface design transition after the Torch Light App button is touched is shown in Figure below.

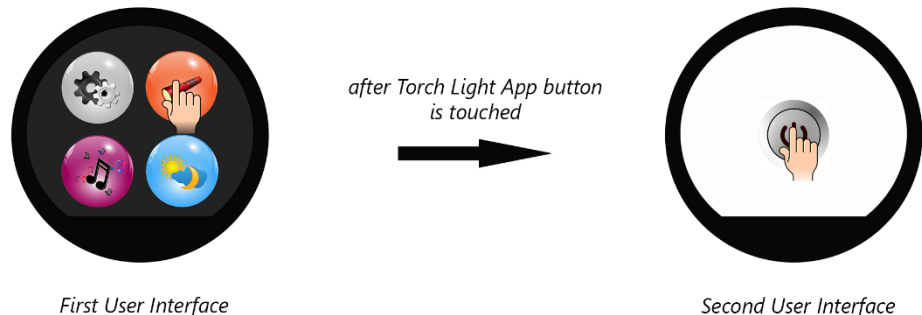


Figure 1.3a: User Interface Design Transition on Touch Light App

2. Setup the CCS IDE development environment by importing the working project from Lab5_1.3.
3. Add the file On.c and Off.c to the project.
4. Complete the code below by referring to the code in Section 1.2. Build to ensure no errors in the code.

```
#include <stdbool.h>
#include <stdint.h>
#include "clock.h"
#include "glib.h"
#include "st7789_spi.h"
#include "it7258_i2c.h"
#include "rgbled.h"

//Include the display driver file
//Include the touch driver file
//Include the rgbled driver file

extern const uint8_t g_pui8NypLogo[];
extern const uint8_t g_pui8SettingsIcon[];
extern const uint8_t g_pui8TorchLightIcon[];
extern const uint8_t g_pui8MusicIcon[];
extern const uint8_t g_pui8WeatherIcon[];

//Insert code from (Exercise 1.2) to declare the On and Off images array

tContext sContext = {};
tRectangle sRect = {};
uint32_t uiDelay;
uint16_t ui16XPosition[3] = {};
uint16_t ui16YPosition[3] = {};
uint8_t ui8Pressure[3] = {};
bool bTouchStatus = 0;
int8_t i8PageCounter = 0;
bool bPagePrinted = 0;

//Insert the TorchLightApp() function code from (Exercise 1.2)

void TouchPanelIntHandler(void)
{
    IT7258IntClear();
    bTouchStatus = 1;
}
```

```

int main(void)
{
    ClockFrequencySet(120000000);           //Set processor clock to 120MHz
    ST7789PinInit(120000000);               //Initialize LCD Controller pin
    IT7258PinInit(120000000);               //Initialize Touch Controller pin
    RGBLEDPinInit();                        //Initialize RGBLED pin
    GrContextInit(&sContext, &g_sST7789);    //Initialize glib Context

    //Register Touch Panel Interrupt Handler
    IT7258IntRegister(TouchPanelIntHandler);

    //Code to display NYP logo, static date and time and unlock button
    GrImageDraw(&sContext, g_pui8NypLogo, 20, 90);
    for(uiDelay=0; uiDelay<7000000; uiDelay++);
    GrContextForegroundSet(&sContext, ClrWhite);
    GrCircleFill(&sContext, 120, 120, 121);
    GrContextForegroundSet(&sContext, ClrBlack);
    GrContextFontSet(&sContext, g_psFontCmss48);
    GrContextBackgroundSet(&sContext, ClrWhite);
    GrContextForegroundSet(&sContext, ClrBlack);
    GrStringDraw(&sContext, "09:55:34", 8, 50, 60, 0);
    GrContextFontSet(&sContext, g_psFontCmss28);
    GrStringDraw(&sContext, "01:04:18", 8, 80, 110, 0);
    sRect.i16XMin = 60;
    sRect.i16YMin = 150;
    sRect.i16XMax = 180;
    sRect.i16YMax = 190;
    GrRectFill(&sContext, &sRect);
    GrLineDraw(&sContext, 58, 148, 182, 148);
    GrLineDraw(&sContext, 58, 148, 58, 192);
    GrLineDraw(&sContext, 58, 192, 182, 192);
    GrLineDraw(&sContext, 182, 148, 182, 192);
    GrContextForegroundSet(&sContext, ClrWhite);
    GrContextFontSet(&sContext, g_psFontCmss14);
    GrStringDraw(&sContext, "PRESS TO UNLOCK", 15, 65, 165, 0);

    while(1)
    {
        if(i8PageCounter == 0)
        {
            if(bTouchStatus == 1)
            {
                //Code to enable touch for UNLOCK button
                IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
                if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
                {
                    if(((60 < ui16XPosition[0]) && (ui16XPosition[0] < 180)) &&
                       ((50 < ui16YPosition[0]) && (ui16YPosition[0] < 90)))
                    {
                        i8PageCounter = 1;
                    }
                }
                while((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
                {
                    IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
                }
                bTouchStatus = 0;
            }
        }
    }
}

```



```

if(i8PageCounter == 1)
{
    if(bPagePrinted == 0)
    {
        //Code to display the 4 App Icons
        GrContextForegroundSet(&sContext, ClrBlack);
        GrCircleFill(&sContext, 120, 120, 121);
        GrImageDraw(&sContext, g_pui8SettingsIcon, 35, 25);
        GrImageDraw(&sContext, g_pui8TorchLightIcon, 125, 25);
        GrImageDraw(&sContext, g_pui8MusicIcon, 35, 115);
        GrImageDraw(&sContext, g_pui8WeatherIcon, 125, 115);
        bPagePrinted = 1;
    }

    if(bTouchStatus == 1)
    {
        //Code to detect the touch points for the 4 App Icons
        GrContextForegroundSet(&sContext, ClrWhite);
        IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);

        if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
        {
            if(((125 < ui16XPosition[0]) && (ui16XPosition[0] < 205)) &&
                ((115 < ui16YPosition[0]) && (ui16YPosition[0] < 195)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
            if(((35 < ui16XPosition[0]) && (ui16XPosition[0] < 115)) &&
                ((115 < ui16YPosition[0]) && (ui16YPosition[0] < 195)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
            //Insert code from (Exercise 1.2) to call the TorchLightApp() Function
        }
        if(((125 < ui16XPosition[0]) && (ui16XPosition[0] < 205)) &&
            ((25 < ui16YPosition[0]) && (ui16YPosition[0] < 105)))
        {
            GrCircleFill(&sContext, 120, 120, 121);
        }
        if(((35 < ui16XPosition[0]) && (ui16XPosition[0] < 115)) &&
            ((25 < ui16YPosition[0]) && (ui16YPosition[0] < 105)))
        {
            GrCircleFill(&sContext, 120, 120, 121);
        }
    }

    while((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
    {
        IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
    }

    bTouchStatus = 0;
}
}
}
}

```

5. Debug and run the code using F8 key.
6. Touch on the Unlock button displayed on the LCD screen to access to the user interface with four App Icons as shown in Figure 1.3a. Continue to touch on Torch Light App icon and an off button is shown, touch to toggle the button on screen to control the RGB LED.